Министерство науки и высшего образования РФ

Отчёт по практикуму

Проектирование и администрирование базы данных учёта экзаменационной сессии

по дисциплине «Системы управления базами данных»

Вариант 7

Выполнил: Лузан Р. Д.

2026 г.

Содержание

Введение **2**

Задание 1.1. Проектирование схемы **2**
Постановка . . . 2
Назначение базы . . . 2
Сущности . . . 2

Задание 1.2. Создание и заполнение таблиц **3**
Постановка . . . 3
DDL . . . 3
Данные . . . 5

Задание 1.3. Операторы манипулирования **7**
Постановка . . . 7
Выборка 1. Нагрузка преподавателя . . . 8
Выборка 2. Дисциплины без сессии . . . 8
Выборка 3. Рейтинг групп . . . 8
Выборка 4. Ближайший контроль . . . 9
Изменение 1. Перевод четвёртого курса . . . 10
Изменение 2. Дифференцированный зачёт . . . 10
Вставка 1. Новая дисциплина и зачёт . . . 11
Вставка 2. Пересдача баз данных . . . 11
Удаление 1. Снятие пересдачи . . . 12
Удаление 2. Дисциплины вне сессии . . . 12

Задание 1.4. Контроль целостности данных **13**
Аномалии транзакций . . . 13
Потерянные изменения (Lost Update) . . . 13
Грязное чтение (Dirty Read) . . . 16
Неповторяющееся чтение (Non-Repeatable Read) . . . 18
Фантомы (Phantom Read) . . . 22
Аномалия сериализации (Serialization Anomaly) . . . 26

Заключение **35**

Введение

Цель практикума — спроектировать базу данных предметной области варианта 7, реализовать её в PostgreSQL и отработать манипулирование данными, транзакции, триггеры, роли, представления и функции PL/pgSQL.

Предметная область — учёт экзаменационной сессии: учебные группы, дисциплины и контрольные мероприятия. СУБД — PostgreSQL 16. Скрипты из каталога `sql` выполнены на этой версии, числовые результаты получены этим прогоном.

Задание 1.1. Проектирование схемы

Постановка

Спроектировать концептуальную схему базы данных своего варианта: сущности, связи, первичные и внешние ключи. Схема должна отвечать задаче предметной области и находиться как минимум в третьей нормальной форме. Типы столбцов на этом шаге не фиксируются.

Назначение базы

База ведётся деканатом. По ней отвечают на вопросы: какие группы учатся на факультете и курсе, какие дисциплины закреплены за кафедрой, когда и в какой форме группа сдаёт дисциплину и кто принимает контроль. Схема хранит состав групп, перечень дисциплин и расписание контрольных мероприятий.

Сущности

- **StudentGroups** — учебная группа. Суррогатный ключ `Id`. Естественный ключ `GroupCode` уникален, но в связи не входит: код группы можно переименовать, не переписывая экзамены. Курс, численность и факультет зависят только от группы.
- **Subjects** — дисциплина. Суррогатный ключ `Id`, уникальное название `SubjectName`. Кафедра и объём часов описывают дисциплину, а не конкретный экзамен.
- **ExamSessions** — ассоциативная сущность «группа сдаёт дисциплину». Ключ `Id`. На группу и дисциплину ссылаются внешние ключи `StudentGroupId` и `SubjectId`. Дата, преподаватель и форма контроля описывают конкретный контроль.

Связи — две связи «один ко многим». Экзамен и есть таблица связи между группой и дисциплиной. Транзитивных зависимостей нет: факультет не определяет курс, кафедра не определяет часы. Схема удовлетворяет третьей нормальной форме. Неключевые атрибуты зависят только от сущности, а не друг от друга.

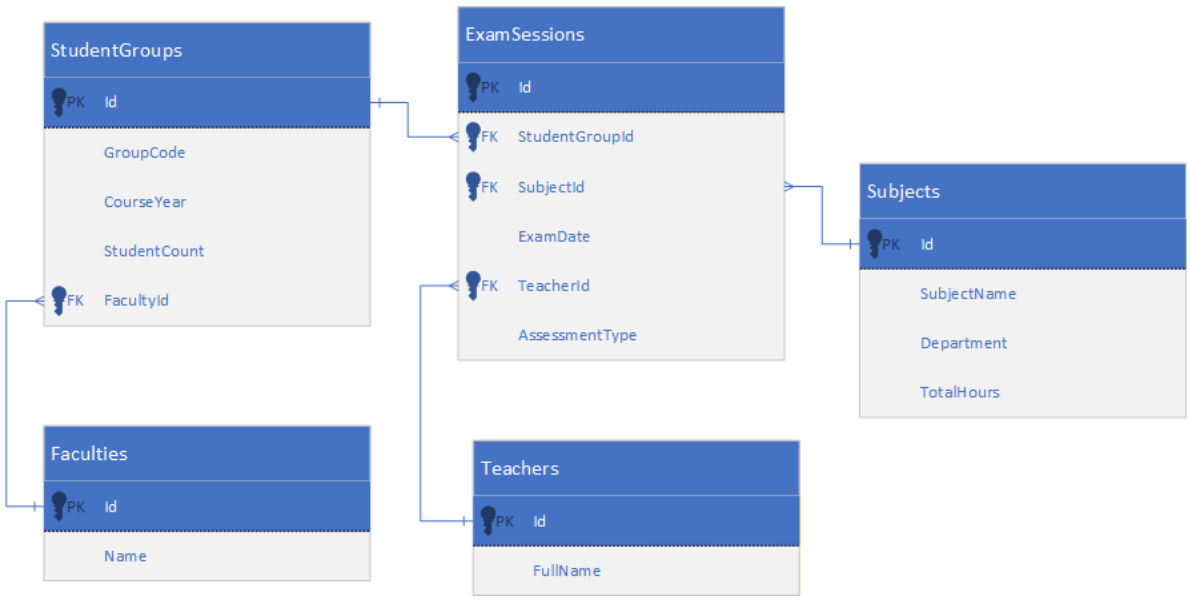


Рис. 1: ER-диаграмма варианта 7

Задание 1.2. Создание и заполнение таблиц

Постановка

По схеме задания 1.1 подготовить DDL с типами, ключами и декларативными ограничениями. Заполнить стержневые сущности не менее чем десятью строками, ассоциативную — не менее чем пятнадцатью. Снять SQL-дамп базы.

DDL

Имена столбцов записаны в PascalCase, как в выданной схеме варианта. Первичные ключи — `UUID`, значение по умолчанию даёт `gen_random_uuid()`. Проверки не дают завести курс вне диапазона 1–6, отрицательную численность, нулевой объём дисциплины и произвольную строку вместо формы контроля. Внешние ключи обновляются и удаляются каскадно. Уникальность тройки группа–дисциплина–дата задана отдельным ограничением: суррогатный ключ повторный экзамен не запрещает. Индексы по обоим внешним ключам ускоряют соединение.

```
-- Практическое задание 1.2. Вариант 7.
-- Предметная область: учёт экзаменационных сессий учебных групп.
-- СУБД: PostgreSQL 16. Суррогатные ключи UUID задаются при вставке.

DROP SCHEMA IF EXISTS exam CASCADE;
CREATE SCHEMA exam;
SET search_path TO exam, public;

-- Стержневая сущность: факультет.
CREATE TABLE "Faculties" (
    "Id"   UUID PRIMARY KEY,
    "Name" VARCHAR(100) NOT NULL UNIQUE
);

-- Стержневая сущность: преподаватель.
CREATE TABLE "Teachers" (
    "Id"       UUID PRIMARY KEY,
```

```
    "FullName" VARCHAR(150) NOT NULL
);

-- Стержневая сущность: учебная группа.
CREATE TABLE "StudentGroups" (
    "Id"           UUID PRIMARY KEY,
    "GroupCode"    VARCHAR(20) NOT NULL UNIQUE,
    "CourseYear"   SMALLINT    NOT NULL CHECK ("CourseYear" BETWEEN 1 AND 6),
    "StudentCount" INTEGER     NOT NULL CHECK ("StudentCount" >= 0),
    "FacultyId"    UUID        NOT NULL,

    CONSTRAINT "FK_StudentGroups_Faculties"
        FOREIGN KEY ("FacultyId") REFERENCES "Faculties" ("Id")
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE INDEX "IX_StudentGroups_FacultyId" ON "StudentGroups" ("FacultyId");

-- Стержневая сущность: учебная дисциплина.
CREATE TABLE "Subjects" (
    "Id"          UUID PRIMARY KEY,
    "SubjectName" VARCHAR(150) NOT NULL UNIQUE,
    "Department"  VARCHAR(100) NOT NULL,
    "TotalHours"  INTEGER      NOT NULL CHECK ("TotalHours" > 0)
);

-- Ассоциативная сущность: экзамен группы по дисциплине в конкретную дату.
CREATE TABLE "ExamSessions" (
    "Id"             UUID PRIMARY KEY,
    "StudentGroupId" UUID        NOT NULL,
    "SubjectId"      UUID        NOT NULL,
    "ExamDate"       DATE        NOT NULL,
    "TeacherId"      UUID        NOT NULL,
    "AssessmentType" VARCHAR(50) NOT NULL
        CHECK ("AssessmentType" IN ('Экзамен', 'Зачёт', 'Дифференцированный зачёт')),

    CONSTRAINT "FK_ExamSessions_StudentGroups"
        FOREIGN KEY ("StudentGroupId") REFERENCES "StudentGroups" ("Id")
        ON UPDATE CASCADE ON DELETE CASCADE,

    CONSTRAINT "FK_ExamSessions_Subjects"
        FOREIGN KEY ("SubjectId") REFERENCES "Subjects" ("Id")
        ON UPDATE CASCADE ON DELETE CASCADE,

    CONSTRAINT "FK_ExamSessions_Teachers"
        FOREIGN KEY ("TeacherId") REFERENCES "Teachers" ("Id")
        ON UPDATE CASCADE ON DELETE RESTRICT,

    CONSTRAINT "UQ_ExamSessions_Group_Subject_Date"
        UNIQUE ("StudentGroupId", "SubjectId", "ExamDate")
);

CREATE INDEX "IX_ExamSessions_StudentGroupId" ON "ExamSessions" ("StudentGroupId");
CREATE INDEX "IX_ExamSessions_SubjectId" ON "ExamSessions" ("SubjectId");
CREATE INDEX "IX_ExamSessions_TeacherId" ON "ExamSessions" ("TeacherId");

COMMENT ON TABLE "Faculties" IS 'Факультет: наименование';
COMMENT ON TABLE "Teachers" IS 'Преподаватель: ФИО';
COMMENT ON TABLE "StudentGroups" IS 'Учебная группа: код, курс, численность, факультет';
COMMENT ON TABLE "Subjects" IS 'Дисциплина: название, кафедра, объём в часах';
```

```
COMMENT ON TABLE "ExamSessions" IS 'Экзамен: группа, дисциплина, дата, преподаватель, форма
    контроля';
```

Данные

Загружены 12 групп, 13 дисциплин и 22 экзамена. Экзамен вставляется не по `UUID`, а соединением с группами и дисциплинами по коду и названию, поэтому скрипт не зависит от сгенерированных ключей. Дисциплина «Философия» загружена без экзамена: по ней строится выборка нераспределённых дисциплин и последующее удаление. Полный скрипт вставки:

```
-- Заполнение схемы exam.
-- UUID задаются явно, база их не генерирует.

SET search_path TO exam, public;

TRUNCATE TABLE
    "ExamSessions",
    "StudentGroups",
    "Subjects",
    "Teachers",
    "Faculties"
RESTART IDENTITY CASCADE;

INSERT INTO "Faculties" ("Id", "Name") VALUES
    ('a1000001-0000-4000-8000-000000000001', 'Институт компьютерных наук'),
    ('a1000001-0000-4000-8000-000000000002', 'Институт прикладной математики'),
    ('a1000001-0000-4000-8000-000000000003', 'Инженерно-экономический институт'),
    ('a1000001-0000-4000-8000-000000000004', 'Механико-технологический институт');

INSERT INTO "Teachers" ("Id", "FullName") VALUES
    ('a2000001-0000-4000-8000-000000000001', 'Соколова Анна Петровна'),
    ('a2000001-0000-4000-8000-000000000002', 'Орлов Игорь Сергеевич'),
    ('a2000001-0000-4000-8000-000000000003', 'Кузнецов Павел Андреевич'),
    ('a2000001-0000-4000-8000-000000000004', 'Белова Мария Олеговна'),
    ('a2000001-0000-4000-8000-000000000005', 'Морозова Елена Викторовна'),
    ('a2000001-0000-4000-8000-000000000006', 'Волков Дмитрий Игоревич'),
    ('a2000001-0000-4000-8000-000000000007', 'Ершов Сергей Павлович'),
    ('a2000001-0000-4000-8000-000000000008', 'Новикова Ольга Дмитриевна');

INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('a3000001-0000-4000-8000-000000000001', 'ИВТ-21', 4, 24, 'a1000001
    -0000-4000-8000-000000000001'),
    ('a3000001-0000-4000-8000-000000000002', 'ИВТ-22', 3, 26, 'a1000001
    -0000-4000-8000-000000000001'),
    ('a3000001-0000-4000-8000-000000000003', 'ИВТ-23', 2, 28, 'a1000001
    -0000-4000-8000-000000000001'),
    ('a3000001-0000-4000-8000-000000000004', 'ПИ-21',  4, 22, 'a1000001
    -0000-4000-8000-000000000001'),
    ('a3000001-0000-4000-8000-000000000005', 'ПИ-22',  3, 25, 'a1000001
    -0000-4000-8000-000000000001'),
    ('a3000001-0000-4000-8000-000000000006', 'ИБ-23',  2, 23, 'a1000001
    -0000-4000-8000-000000000001'),
    ('a3000001-0000-4000-8000-000000000007', 'ПМ-21',  4, 20, 'a1000001
    -0000-4000-8000-000000000002'),
    ('a3000001-0000-4000-8000-000000000008', 'ПМ-22',  3, 21, 'a1000001
    -0000-4000-8000-000000000002'),
```

```
    ('a3000001-0000-4000-8000-000000000009', 'ЭК-21',  4, 30, 'a1000001
    -0000-4000-8000-000000000003'),
    ('a3000001-0000-4000-8000-00000000000a', 'ЭК-22',  3, 27, 'a1000001
    -0000-4000-8000-000000000003'),
    ('a3000001-0000-4000-8000-00000000000b', 'МТ-21',  4, 18, 'a1000001
    -0000-4000-8000-000000000004'),
    ('a3000001-0000-4000-8000-00000000000c', 'МТ-22',  3, 19, 'a1000001
    -0000-4000-8000-000000000004');

INSERT INTO "Subjects" ("Id", "SubjectName", "Department", "TotalHours") VALUES
    ('a4000001-0000-4000-8000-000000000001', 'Базы␣данных',                  'Кафедра␣вычисл
    ительных␣систем',     144),
    ('a4000001-0000-4000-8000-000000000002', 'Операционные␣системы',         'Кафедра␣вычисл
    ительных␣систем',     108),
    ('a4000001-0000-4000-8000-000000000003', 'Компьютерные␣сети',            'Кафедра␣вычисл
    ительных␣систем',     108),
    ('a4000001-0000-4000-8000-000000000004', 'Программирование',             'Кафедра␣програ
    ммного␣обеспечения',  144),
    ('a4000001-0000-4000-8000-000000000005', 'Информационная␣безопасность',  'Кафедра␣защиты
    ␣информации',         108),
    ('a4000001-0000-4000-8000-000000000006', 'Алгоритмы␣и␣структуры␣данных', 'Кафедра␣прикла
    дной␣математики',     108),
    ('a4000001-0000-4000-8000-000000000007', 'Дискретная␣математика',        'Кафедра␣прикла
    дной␣математики',      72),
    ('a4000001-0000-4000-8000-000000000008', 'Математический␣анализ',        'Кафедра␣прикла
    дной␣математики',     144),
    ('a4000001-0000-4000-8000-000000000009', 'Экономика␣предприятия',        'Кафедра␣эконом
    ики',                 108),
    ('a4000001-0000-4000-8000-00000000000a', 'Бухгалтерский␣учёт',           'Кафедра␣эконом
    ики',                  72),
    ('a4000001-0000-4000-8000-00000000000b', 'Технология␣машиностроения',    'Кафедра␣технол
    огии␣машиностроения', 144),
    ('a4000001-0000-4000-8000-00000000000c', 'Материаловедение',             'Кафедра␣технол
    огии␣машиностроения',  72),
    ('a4000001-0000-4000-8000-00000000000d', 'Философия',                    'Кафедра␣гумани
    тарных␣наук',          72);

INSERT INTO "ExamSessions"
    ("Id", "StudentGroupId", "SubjectId", "ExamDate", "TeacherId", "AssessmentType")
VALUES
    ('a5000001-0000-4000-8000-000000000001', 'a3000001-0000-4000-8000-000000000001', '
    a4000001-0000-4000-8000-000000000001', DATE '2026-01-12', 'a2000001
    -0000-4000-8000-000000000001', 'Экзамен'),
    ('a5000001-0000-4000-8000-000000000002', 'a3000001-0000-4000-8000-000000000001', '
    a4000001-0000-4000-8000-000000000002', DATE '2026-01-19', 'a2000001
    -0000-4000-8000-000000000003', 'Экзамен'),
    ('a5000001-0000-4000-8000-000000000003', 'a3000001-0000-4000-8000-000000000001', '
    a4000001-0000-4000-8000-000000000004', DATE '2026-01-09', 'a2000001
    -0000-4000-8000-000000000005', 'Экзамен'),
    ('a5000001-0000-4000-8000-000000000004', 'a3000001-0000-4000-8000-000000000002', '
    a4000001-0000-4000-8000-000000000001', DATE '2026-01-14', 'a2000001
    -0000-4000-8000-000000000001', 'Экзамен'),
    ('a5000001-0000-4000-8000-000000000005', 'a3000001-0000-4000-8000-000000000002', '
    a4000001-0000-4000-8000-000000000003', DATE '2026-01-18', 'a2000001
    -0000-4000-8000-000000000002', 'Экзамен'),
    ('a5000001-0000-4000-8000-000000000006', 'a3000001-0000-4000-8000-000000000002', '
    a4000001-0000-4000-8000-000000000008', DATE '2026-01-11', 'a2000001
    -0000-4000-8000-000000000006', 'Экзамен'),
    ('a5000001-0000-4000-8000-000000000007', 'a3000001-0000-4000-8000-000000000003', '
    a4000001-0000-4000-8000-000000000004', DATE '2026-06-05', 'a2000001
```

```
-0000-4000-8000-000000000005', 'Дифференцированный␣зачёт'),
('a5000001-0000-4000-8000-000000000008', 'a3000001-0000-4000-8000-000000000003', '
a4000001-0000-4000-8000-000000000001', DATE '2026-06-10', 'a2000001
-0000-4000-8000-000000000001', 'Зачёт'),
('a5000001-0000-4000-8000-000000000009', 'a3000001-0000-4000-8000-000000000003', '
a4000001-0000-4000-8000-000000000007', DATE '2026-06-08', 'a2000001
-0000-4000-8000-000000000006', 'Экзамен'),
('a5000001-0000-4000-8000-00000000000a', 'a3000001-0000-4000-8000-000000000004', '
a4000001-0000-4000-8000-000000000001', DATE '2026-01-16', 'a2000001
-0000-4000-8000-000000000002', 'Экзамен'),
('a5000001-0000-4000-8000-00000000000b', 'a3000001-0000-4000-8000-000000000004', '
a4000001-0000-4000-8000-000000000003', DATE '2026-01-13', 'a2000001
-0000-4000-8000-000000000002', 'Экзамен'),
('a5000001-0000-4000-8000-00000000000c', 'a3000001-0000-4000-8000-000000000005', '
a4000001-0000-4000-8000-000000000002', DATE '2026-01-21', 'a2000001
-0000-4000-8000-000000000003', 'Дифференцированный␣зачёт'),
('a5000001-0000-4000-8000-00000000000d', 'a3000001-0000-4000-8000-000000000006', '
a4000001-0000-4000-8000-000000000002', DATE '2026-01-23', 'a2000001
-0000-4000-8000-000000000003', 'Зачёт'),
('a5000001-0000-4000-8000-00000000000e', 'a3000001-0000-4000-8000-000000000007', '
a4000001-0000-4000-8000-000000000006', DATE '2026-01-26', 'a2000001
-0000-4000-8000-000000000005', 'Экзамен'),
('a5000001-0000-4000-8000-00000000000f', 'a3000001-0000-4000-8000-000000000007', '
a4000001-0000-4000-8000-000000000008', DATE '2026-06-15', 'a2000001
-0000-4000-8000-000000000006', 'Зачёт'),
('a5000001-0000-4000-8000-000000000010', 'a3000001-0000-4000-8000-000000000007', '
a4000001-0000-4000-8000-000000000007', DATE '2026-06-08', 'a2000001
-0000-4000-8000-000000000006', 'Экзамен'),
('a5000001-0000-4000-8000-000000000011', 'a3000001-0000-4000-8000-000000000008', '
a4000001-0000-4000-8000-000000000007', DATE '2026-01-15', 'a2000001
-0000-4000-8000-000000000006', 'Экзамен'),
('a5000001-0000-4000-8000-000000000012', 'a3000001-0000-4000-8000-000000000009', '
a4000001-0000-4000-8000-000000000009', DATE '2026-01-22', 'a2000001
-0000-4000-8000-000000000004', 'Экзамен'),
('a5000001-0000-4000-8000-000000000013', 'a3000001-0000-4000-8000-000000000009', '
a4000001-0000-4000-8000-00000000000a', DATE '2026-06-06', 'a2000001
-0000-4000-8000-000000000004', 'Экзамен'),
('a5000001-0000-4000-8000-000000000014', 'a3000001-0000-4000-8000-00000000000a', '
a4000001-0000-4000-8000-00000000000a', DATE '2026-01-27', 'a2000001
-0000-4000-8000-000000000004', 'Дифференцированный␣зачёт'),
('a5000001-0000-4000-8000-000000000015', 'a3000001-0000-4000-8000-00000000000b', '
a4000001-0000-4000-8000-00000000000b', DATE '2026-01-29', 'a2000001
-0000-4000-8000-000000000007', 'Экзамен'),
('a5000001-0000-4000-8000-000000000016', 'a3000001-0000-4000-8000-00000000000c', '
a4000001-0000-4000-8000-00000000000b', DATE '2026-01-30', 'a2000001
-0000-4000-8000-000000000007', 'Экзамен');
```

После загрузки: `StudentGroups` — 12 строк, `Subjects` — 13, `ExamSessions` — 22.

Задание 1.3. Операторы манипулирования

Постановка

Подготовить четыре осмысленные выборки и по два запроса на вставку, изменение и удаление. В запросах должны встретиться соединение, обобщённое табличное выражение, оконная функция, агрегирование, группировка, вложенный и коррелированный подзапросы.

Выборка 1. Нагрузка преподавателя

Зимняя сессия считается через CTE. К экзамену присоединяется численность группы по `StudentGroupId`, преподаватели группируются, в выборку попадают те, у кого не меньше двух контролей. Оконная функция считает долю преподавателя во всей сессии, `ROUND` оставляет три знака.

```
WITH winter AS (
    SELECT * FROM "ExamSessions"
    WHERE "ExamDate" >= DATE '2026-01-01'
      AND "ExamDate" <  DATE '2026-02-01'
)
SELECT w."TeacherFullName" AS "Преподаватель",
       COUNT(*) AS "Экзаменов",
       COUNT(DISTINCT w."StudentGroupId") AS "Групп",
       SUM(g."StudentCount") AS "Студентов",
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 3)
           AS "ДоляСессииПроцент"
FROM winter w
JOIN "StudentGroups" g ON g."Id" = w."StudentGroupId"
GROUP BY w."TeacherFullName"
HAVING COUNT(*) >= 2
ORDER BY "Студентов" DESC;
```

Таблица 1: Нагрузка преподавателей, январь 2026

Преподаватель	Экзаменов	Групп	Студентов	Доля, %
Кузнецов П. А.	3	3	75	18,750
Орлов И. С.	3	2	70	18,750
Белова М. О.	2	2	57	12,500
Соколова А. П.	2	2	50	12,500
Морозова Е. В.	2	2	44	12,500
Волков Д. И.	2	2	41	12,500
Ершов С. П.	2	2	37	12,500

Семь строк дают $3+3+2+2+2+2+2 = 16$ экзаменов. Доли сходятся: $100 \cdot 16/16 = 100$.

Выборка 2. Дисциплины без сессии

Левое соединение оставляет дисциплины, которым не нашлось экзамена. Коррелированный подзапрос считает средний объём кафедры.

```
SELECT s."SubjectName", s."Department", s."TotalHours",
       (SELECT ROUND(AVG(s2."TotalHours"), 3) FROM "Subjects" s2
        WHERE s2."Department" = s."Department") AS "СредниеЧасыКафедры"
FROM "Subjects" s
LEFT JOIN "ExamSessions" e ON e."SubjectId" = s."Id"
WHERE e."SubjectId" IS NULL;
```

Результат — одна строка: Философия, кафедра гуманитарных наук, 72 часа. Среднее по кафедре тоже 72, дисциплина на кафедре единственная.

Выборка 3. Рейтинг групп

CTE считает число контролей по `StudentGroupId`. `ROW_NUMBER() OVER (PARTITION BY "Faculty")` расставляет места отдельно внутри факультета. Первые места внутри факультета: ЭК-21

(2 контроля), ИВТ-21 (3), ПМ-21 (3), МТ-21 (1). Всего 12 строк, по одной на группу: левое соединение сохраняет группы без экзаменов.

```
WITH load AS (
    SELECT g."Faculty", g."GroupCode", g."CourseYear",
           COUNT(e."ExamDate") AS "ЧислоКонтролей",
           ROUND(COUNT(e."ExamDate")::numeric
               / NULLIF(g."StudentCount", 0), 3) AS "КонтролейНаСтудента"
    FROM "StudentGroups" g
    LEFT JOIN "ExamSessions" e ON e."StudentGroupId" = g."Id"
    GROUP BY g."Faculty", g."GroupCode", g."CourseYear", g."StudentCount"
)
SELECT "Faculty" AS "Факультет", "GroupCode" AS "Группа",
       "CourseYear" AS "Курс", "ЧислоКонтролей", "КонтролейНаСтудента",
       ROW_NUMBER() OVER (
           PARTITION BY "Faculty"
           ORDER BY "ЧислоКонтролей" DESC, "GroupCode"
       ) AS "МестоНаФакультете"
FROM load
ORDER BY "Факультет", "МестоНаФакультете";
```

Таблица 2: Вывод выборки 3, 12 строк

Факультет	Группа	Курс	Контролей	На студента	Место
Инженерно-экономический институт	ЭК-21	4	2	0,067	1
Инженерно-экономический институт	ЭК-22	3	1	0,037	2
Институт компьютерных наук	ИВТ-21	4	3	0,125	1
Институт компьютерных наук	ИВТ-22	3	3	0,115	2
Институт компьютерных наук	ИВТ-23	2	3	0,107	3
Институт компьютерных наук	ПИ-21	4	2	0,091	4
Институт компьютерных наук	ИБ-23	2	1	0,043	5
Институт компьютерных наук	ПИ-22	3	1	0,040	6
Институт прикладной математики	ПМ-21	4	3	0,150	1
Институт прикладной математики	ПМ-22	3	1	0,048	2
Механико-технологический институт	МТ-21	4	1	0,056	1
Механико-технологический институт	МТ-22	3	1	0,053	2

Выборка 4. Ближайший контроль

Коррелированный `LATERAL` берёт для группы один экзамен с минимальной датой строго позже 15 января 2026 и подставляет название дисциплины по `SubjectId`.

```
SELECT g."GroupCode" AS "Группа",
       nxt."SubjectName" AS "БлижайшаяДисциплина",
       nxt."ExamDate" AS "Дата",
       nxt."TeacherFullName" AS "Преподаватель",
       nxt."AssessmentType" AS "Форма"
FROM "StudentGroups" g
JOIN LATERAL (
    SELECT s."SubjectName", e."ExamDate",
           e."TeacherFullName", e."AssessmentType"
    FROM "ExamSessions" e
    JOIN "Subjects" s ON s."Id" = e."SubjectId"
    WHERE e."StudentGroupId" = g."Id"
      AND e."ExamDate" > DATE '2026-01-15'
    ORDER BY e."ExamDate"
    LIMIT 1
) nxt ON TRUE
```

```
ORDER BY nxt."ExamDate", g."GroupCode";
```

Таблица 3: Вывод выборки 4, 11 строк

Группа	Дата	Дисциплина	Форма	Преподаватель
ПИ-21	16.01	Базы данных	Экзамен	Орлов И. С.
ИВТ-22	18.01	Компьютерные сети	Экзамен	Орлов И. С.
ИВТ-21	19.01	Операционные системы	Экзамен	Кузнецов П. А.
ПИ-22	21.01	Операционные системы	Диф. зачёт	Кузнецов П. А.
ЭК-21	22.01	Экономика предприятия	Экзамен	Белова М. О.
ПМ-21	26.01	Алгоритмы и структуры данных	Экзамен	Морозова Е. В.
ЭК-22	27.01	Бухгалтерский учёт	Диф. зачёт	Белова М. О.
МТ-21	29.01	Технология машиностроения	Экзамен	Ершов С. П.
МТ-22	30.01	Технология машиностроения	Экзамен	Ершов С. П.
ИВТ-23	05.06	Программирование	Диф. зачёт	Морозова Е. В.
ИБ-23	12.06	Информационная безопасность	Экзамен	Новикова О. Д.

Одиннадцать строк. ПМ-22 в результат не попадает: её экзамен по дискретной математике стоит ровно 15 января, а условие строгое.

Изменение 1. Перевод четвёртого курса

Группы 4 курса института компьютерных наук переводятся на следующий курс. Численность уменьшается на двух отчисленных и не опускается ниже нуля.

```
UPDATE "StudentGroups"
SET "CourseYear"   = "CourseYear" + 1,
    "StudentCount" = GREATEST("StudentCount" - 2, 0)
WHERE "CourseYear" = 4
  AND "Faculty" = 'Институт␣компьютерных␣наук'
RETURNING "GroupCode", "CourseYear", "StudentCount";
```

До изменения условию отвечали ИВТ-21 (4 курс, 24) и ПИ-21 (4 курс, 22). Вывод оператора:

```
UPDATE 2
 GroupCode | CourseYear | StudentCount
-----------+------------+--------------
 ИВТ-21    |          5 |           22
 ПИ-21     |          5 |           20
```

Повторный отбор групп института оставляет остальные четыре строки прежними: ИВТ-22 (3 курс, 26), ПИ-22 (3 курс, 25), ИБ-23 (2 курс, 23), ИВТ-23 (2 курс, 28).

Изменение 2. Дифференцированный зачёт

Зачёты Соколовой по кафедре вычислительных систем становятся дифференцированными.

```
UPDATE "ExamSessions" e
SET "AssessmentType" = 'Дифференцированный␣зачёт'
FROM "Subjects" s
WHERE e."SubjectId" = s."Id"
  AND s."Department" = 'Кафедра␣вычислительных␣систем'
  AND e."TeacherFullName" = 'Соколова␣Анна␣Петровна'
  AND e."AssessmentType" = 'Зачёт'
RETURNING s."SubjectName", e."ExamDate", e."AssessmentType";
```

До изменения условию отвечала одна строка: ИВТ-23, базы данных, 10 июня 2026, зачёт. Вывод оператора:

```
UPDATE 1
 SubjectName |  ExamDate  |      AssessmentType
-------------+------------+--------------------------
 Базы данных | 2026-06-10 | Дифференцированный зачёт
```

Январские экзамены Соколовой не затронуты: у ИВТ-21 (12 января) и ИВТ-22 (14 января) форма «Экзамен», а не «Зачёт».

Вставка 1. Новая дисциплина и зачёт

В перечень добавляется дисциплина «Криптографические методы» и первый зачёт по ней для группы ИБ-23.

```
INSERT INTO "Subjects" ("SubjectName", "Department", "TotalHours")
VALUES ('Криптографические␣методы', 'Кафедра␣защиты␣информации', 72);

INSERT INTO "ExamSessions"
    ("StudentGroupId", "SubjectId", "ExamDate",
     "TeacherFullName", "AssessmentType")
SELECT g."Id", s."Id", DATE '2026-06-18',
       'Новикова␣Ольга␣Дмитриевна', 'Зачёт'
FROM "StudentGroups" g
JOIN "Subjects" s ON s."SubjectName" = 'Криптографические␣методы'
WHERE g."GroupCode" = 'ИБ-23';
```

Оба оператора вернули `INSERT 0 1`. Проверочная выборка по названию дисциплины дала одну строку:

```
 GroupCode | SubjectName              | Department
-----------+--------------------------+---------------------------
 ИБ-23     | Криптографические методы | Кафедра защиты информации

 TotalHours | ExamDate   | TeacherFullName           | AssessmentType
------------+------------+---------------------------+----------------
         72 | 2026-06-18 | Новикова Ольга Дмитриевна | Зачёт
```

Вставка 2. Пересдача баз данных

Январский экзамен по базам данных назначается повторно через 30 дней. Дата берётся из уже существующей строки.

```
INSERT INTO "ExamSessions"
    ("StudentGroupId", "SubjectId", "ExamDate",
     "TeacherFullName", "AssessmentType")
SELECT e."StudentGroupId", e."SubjectId", e."ExamDate" + 30,
       e."TeacherFullName", 'Экзамен'
FROM "ExamSessions" e
JOIN "Subjects" s ON s."Id" = e."SubjectId"
WHERE s."SubjectName" = 'Базы␣данных'
  AND e."AssessmentType" = 'Экзамен'
  AND e."ExamDate" < DATE '2026-02-01';
```

До вставки по базам данных было четыре строки: три январских экзамена и июньский дифференцированный зачёт ИВТ-23. Оператор вернул `INSERT 0 3`. Повторный отбор дал семь строк:

```
 GroupCode | ExamDate   | TeacherFullName         | AssessmentType
-----------+------------+-------------------------+----------------
 ИВТ-21    | 2026-01-12 | Соколова Анна Петровна | Экзамен
 ИВТ-21    | 2026-02-11 | Соколова Анна Петровна | Экзамен
 ИВТ-22    | 2026-01-14 | Соколова Анна Петровна | Экзамен
 ИВТ-22    | 2026-02-13 | Соколова Анна Петровна | Экзамен
 ИВТ-23    | 2026-06-10 | Соколова Анна Петровна | Диф. зачёт
 ПИ-21     | 2026-01-16 | Орлов Игорь Сергеевич  | Экзамен
 ПИ-21     | 2026-02-15 | Орлов Игорь Сергеевич  | Экзамен
```

Июньский зачёт ИВТ-23 не продублирован: фильтр требует форму «Экзамен» и дату раньше 1 февраля.

Удаление 1. Снятие пересдачи

Пересдача снимается, если по той же паре группа–дисциплина уже есть более ранний экзамен. Проверка идёт коррелированным `EXISTS`.

```
DELETE FROM "ExamSessions" e
USING "Subjects" s
WHERE e."SubjectId" = s."Id"
  AND s."SubjectName" = 'Базы␣данных'
  AND e."ExamDate" >= DATE '2026-02-01'
  AND EXISTS (
      SELECT 1
      FROM "ExamSessions" earlier
      WHERE earlier."StudentGroupId" = e."StudentGroupId"
        AND earlier."SubjectId" = e."SubjectId"
        AND earlier."ExamDate" < e."ExamDate"
  );
```

Оператор вернул `DELETE 3`: сняты 11, 13 и 15 февраля. Повторный отбор снова даёт четыре исходные строки:

```
 GroupCode | ExamDate   | TeacherFullName         | AssessmentType
-----------+------------+-------------------------+---------------------------
 ИВТ-21    | 2026-01-12 | Соколова Анна Петровна | Экзамен
 ИВТ-22    | 2026-01-14 | Соколова Анна Петровна | Экзамен
 ИВТ-23    | 2026-06-10 | Соколова Анна Петровна | Дифференцированный зачёт
 ПИ-21     | 2026-01-16 | Орлов Игорь Сергеевич  | Экзамен
```

Удаление 2. Дисциплины вне сессии

Удаляются дисциплины, которым не соответствует ни одна строка экзамена.

```
DELETE FROM "Subjects" s
WHERE NOT EXISTS (
    SELECT 1 FROM "ExamSessions" e WHERE e."SubjectId" = s."Id"
)
RETURNING s."SubjectName", s."Department";
```

До удаления таким был единственный предмет. Вывод оператора:

```
DELETE 1
 SubjectName | Department
-------------+---------------------------
 Философия   | Кафедра гуманитарных наук
```

Криптографические методы сохранены: на них ссылается зачёт ИБ-23. После удаления в таблице 13 дисциплин, и каждая встречается в сессии.

Задание 1.4. Контроль целостности данных

Аномалии транзакций

Проверки выполнены в PostgreSQL 16.15 в двух независимых сеансах A и B. Ниже приведены команды в хронологическом порядке и полученные результаты. Использована отдельная тестовая копия структуры таблицы `exam."StudentGroups"`, доступная обоим сеансам через `search_path` под именем `"StudentGroups"`. Внешние ключи и триггеры исходной таблицы в копию не переносились. Основные учебные данные не изменялись. Тестовые коды групп: IVT-21 и IVT-22.

Потерянные изменения (Lost Update)

Проверялось сохранение двух SQL-инкрементов: A прибавляет 5, B прибавляет 3. Ожидаемая численность после обеих операций: $24 + 5 + 3 = 32$.

Проверка на READ COMMITTED.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем READ COMMITTED.

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

Сеанс A. Результат: read committed.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем READ COMMITTED.

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

Сеанс B. Результат: read committed.

```
SHOW transaction_isolation;
```

Сеанс A. Первое чтение: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Чтение до фиксации A: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. Обновлена одна строка. Внутри A численность равна 29; COMMIT ещё не выполнен.

```
UPDATE "StudentGroups" SET "StudentCount" = "StudentCount" + 5
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Запрос отправлен и ожидает блокировку строки; ответ UPDATE ещё не получен.

```
UPDATE "StudentGroups" SET "StudentCount" = "StudentCount" + 3
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. A зафиксирована, блокировка освобождена.

```
COMMIT;
```

Сеанс B. Ожидавший UPDATE завершился: обновлена одна строка. Прибавка 3 применена к актуальному значению 29, получено 32.

Сеанс B. B зафиксирована.

```
COMMIT;
```

Сеанс A. Итоговая проверка после обеих фиксаций: 32.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Вывод для READ COMMITTED: потерянного изменения нет; обе прибавки сохранены.

Проверка на READ UNCOMMITTED.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем READ UNCOMMITTED.

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
```

Сеанс A. Результат: read uncommitted.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем READ UNCOMMITTED.

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
```

Сеанс B. Результат: read uncommitted.

```
SHOW transaction_isolation;
```

Сеанс A. Первое чтение: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Чтение до фиксации A: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. Обновлена одна строка. Внутри A численность равна 29; COMMIT ещё не выполнен.

```
UPDATE "StudentGroups" SET "StudentCount" = "StudentCount" + 5
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Запрос отправлен и ожидает блокировку строки; ответ UPDATE ещё не получен.

```
UPDATE "StudentGroups" SET "StudentCount" = "StudentCount" + 3
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. A зафиксирована, блокировка освобождена.

```
COMMIT;
```

Сеанс B. Ожидавший UPDATE завершился: обновлена одна строка. Прибавка 3 применена к актуальному значению 29, получено 32.

Сеанс B. B зафиксирована.

```
COMMIT;
```

Сеанс A. Итоговая проверка после обеих фиксаций: 32.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Вывод для READ UNCOMMITTED: потерянного изменения нет; обе прибавки сохранены.

Грязное чтение (Dirty Read)

Проверялась видимость незафиксированного изменения другой транзакции. А записывает 999 вместо 24, затем отменяет изменение.

Проверка на READ COMMITTED.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем READ COMMITTED.

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

Сеанс A. Результат: read committed.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем READ COMMITTED.

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

Сеанс B. Результат: read committed.

```
SHOW transaction_isolation;
```

Сеанс A. Обновлена одна строка; COMMIT не выполняется.

```
UPDATE "StudentGroups" SET "StudentCount" = 999
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Чтение при незавершённой A: 24, а не 999.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. Незафиксированное значение 999 отменено.

```
ROLLBACK;
```

Сеанс B. Повторное чтение после отката A: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Транзакция чтения завершена.

```
COMMIT;
```

Сеанс A. Итоговая проверка: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Вывод для READ COMMITTED: грязного чтения нет; значение 999 другому сеансу недоступно.

Проверка на READ UNCOMMITTED.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем READ UNCOMMITTED.

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
```

Сеанс A. Результат: read uncommitted.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем READ UNCOMMITTED.

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
```

Сеанс B. Результат: read uncommitted.

```
SHOW transaction_isolation;
```

Сеанс A. Обновлена одна строка; COMMIT не выполняется.

```
UPDATE "StudentGroups" SET "StudentCount" = 999
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Чтение при незавершённой A: 24, а не 999.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. Незафиксированное значение 999 отменено.

```
ROLLBACK;
```

Сеанс B. Повторное чтение после отката A: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Транзакция чтения завершена.

```
COMMIT;
```

Сеанс A. Итоговая проверка: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Вывод для READ UNCOMMITTED: грязного чтения нет; значение 999 другому сеансу недоступно.

Неповторяющееся чтение (Non-Repeatable Read)

Проверялось повторное чтение одной строки в A до и после фиксации изменения в B.

Проверка на READ COMMITTED.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем READ COMMITTED.

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

Сеанс A. Результат: read committed.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем READ COMMITTED.

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

Сеанс B. Результат: read committed.

```
SHOW transaction_isolation;
```

Сеанс A. Первое чтение: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Обновлена одна строка; новое значение 30 ещё не зафиксировано.

```
UPDATE "StudentGroups" SET "StudentCount" = 30
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Значение 30 зафиксировано.

```
COMMIT;
```

Сеанс A. Второе чтение внутри той же транзакции: 30.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. Транзакция A завершена.

```
COMMIT;
```

Сеанс A. Чтение после завершения A: 30; изменение B сохранено.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Вывод для READ COMMITTED: неповторяющееся чтение наблюдается: внутри A значение изменилось с 24 на 30.

Проверка на READ UNCOMMITTED.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем READ UNCOMMITTED.

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
```

Сеанс A. Результат: read uncommitted.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем READ UNCOMMITTED.

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
```

Сеанс B. Результат: read uncommitted.

```
SHOW transaction_isolation;
```

Сеанс A. Первое чтение: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Обновлена одна строка; новое значение 30 ещё не зафиксировано.

```
UPDATE "StudentGroups" SET "StudentCount" = 30
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Значение 30 зафиксировано.

```
COMMIT;
```

Сеанс A. Второе чтение внутри той же транзакции: 30.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. Транзакция A завершена.

```
COMMIT;
```

Сеанс A. Чтение после завершения A: 30; изменение B сохранено.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Вывод для READ UNCOMMITTED: неповторяющееся чтение наблюдается: внутри A значение изменилось с 24 на 30.

Проверка на REPEATABLE READ.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс A. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс B. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс A. Первое чтение: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Обновлена одна строка; новое значение 30 ещё не зафиксировано.

```
UPDATE "StudentGroups" SET "StudentCount" = 30
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Значение 30 зафиксировано.

```
COMMIT;
```

Сеанс A. Второе чтение внутри той же транзакции: 24.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. Транзакция A завершена.

```
COMMIT;
```

Сеанс A. Чтение после завершения A: 30; изменение B сохранено.

```
SELECT "StudentCount" FROM "StudentGroups"
WHERE "GroupCode" = 'IVT-21';
```

Вывод для REPEATABLE READ: неповторяющегося чтения нет: оба чтения внутри A дали 24.

Фантомы (Phantom Read)

Проверялось изменение количества групп четвёртого курса при повторном выполнении одного условия в транзакции A.

Проверка на READ COMMITTED.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем READ COMMITTED.

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

Сеанс A. Результат: read committed.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем READ COMMITTED.

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

Сеанс B. Результат: read committed.

```
SHOW transaction_isolation;
```

Сеанс A. Первый подсчёт: 2.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Сеанс B. Вставлена одна новая группа четвёртого курса; вставка пока не зафиксирована.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES ('00000000-0000-0000-0000-000000000003', 'TEST-4', 4, 20,
        '00000000-0000-0000-0000-000000000010');
```

Сеанс B. Вставка зафиксирована.

```
COMMIT;
```

Сеанс A. Второй подсчёт внутри той же транзакции: 3.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Сеанс A. Транзакция A завершена.

```
COMMIT;
```

Сеанс A. Подсчёт после завершения A: 3; вставленная строка сохранена.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Вывод для READ COMMITTED: фантомное чтение наблюдается: количество строк внутри A изменилось с 2 на 3.

Проверка на READ UNCOMMITTED.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем READ UNCOMMITTED.

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
```

Сеанс A. Результат: read uncommitted.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем READ UNCOMMITTED.

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
```

Сеанс B. Результат: read uncommitted.

```
SHOW transaction_isolation;
```

Сеанс A. Первый подсчёт: 2.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Сеанс B. Вставлена одна новая группа четвёртого курса; вставка пока не зафиксирована.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES ('00000000-0000-0000-0000-000000000003', 'TEST-4', 4, 20,
        '00000000-0000-0000-0000-000000000010');
```

Сеанс B. Вставка зафиксирована.

```
COMMIT;
```

Сеанс A. Второй подсчёт внутри той же транзакции: 3.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Сеанс A. Транзакция A завершена.

```
COMMIT;
```

Сеанс A. Подсчёт после завершения A: 3; вставленная строка сохранена.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Вывод для READ UNCOMMITTED: фантомное чтение наблюдается: количество строк внутри A изменилось с 2 на 3.

Проверка на REPEATABLE READ.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс A. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс B. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс A. Первый подсчёт: 2.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Сеанс B. Вставлена одна новая группа четвёртого курса; вставка пока не зафиксирована.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES ('00000000-0000-0000-0000-000000000003', 'TEST-4', 4, 20,
        '00000000-0000-0000-0000-000000000010');
```

Сеанс B. Вставка зафиксирована.

```
COMMIT;
```

Сеанс A. Второй подсчёт внутри той же транзакции: 2.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Сеанс A. Транзакция A завершена.

```
COMMIT;
```

Сеанс A. Подсчёт после завершения A: 3; вставленная строка сохранена.

```
SELECT COUNT(*) FROM "StudentGroups" WHERE "CourseYear" = 4;
```

Вывод для REPEATABLE READ: фантомного чтения нет: оба подсчёта внутри A дали 2.

Аномалия сериализации (Serialization Anomaly)

A копирует численность IVT-22 в IVT-21, B копирует численность IVT-21 в IVT-22. Для каждого уровня проверены оба последовательных порядка и параллельное выполнение. Исходная пара равна $24, 26$; порядок значений в паре: IVT-21, IVT-22.

Уровень REPEATABLE READ. Последовательное выполнение A, затем B.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс A. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс A. Обновлена одна строка: в IVT-21 записано 26.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-22')
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. А зафиксирована.

```
COMMIT;
```

Сеанс B. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс B. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс B. Прочитано зафиксированное значение IVT-21: 26. В IVT-22 записано 26.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-21')
WHERE "GroupCode" = 'IVT-22';
```

Сеанс B. В зафиксирована.

```
COMMIT;
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 26
 IVT-22    | 26
```

Уровень REPEATABLE READ. Последовательное выполнение B, затем A.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс B. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс B. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс B. Обновлена одна строка: в IVT-22 записано 24.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-21')
WHERE "GroupCode" = 'IVT-22';
```

Сеанс B. В зафиксирована.

```
COMMIT;
```

Сеанс A. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс A. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс A. Прочитано зафиксированное значение IVT-22: 24. В IVT-21 записано 24.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-22')
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. А зафиксирована.

```
COMMIT;
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 24
```

Уровень REPEATABLE READ. Параллельное выполнение.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс A. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем REPEATABLE READ.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

Сеанс B. Результат: repeatable read.

```
SHOW transaction_isolation;
```

Сеанс A. Обновлена одна строка: IVT-21 получает 26. A пока не зафиксирована.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-22')
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Обновлена другая строка: B читает старое зафиксированное значение IVT-21, равное 24, и записывает 24 в IVT-22. B пока не зафиксирована.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-21')
WHERE "GroupCode" = 'IVT-22';
```

Сеанс A. A успешно зафиксирована.

```
COMMIT;
```

Сеанс B. B успешно зафиксирована.

```
COMMIT;
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 26
 IVT-22    | 24
```

Вывод для REPEATABLE READ: получено $(26, 24)$, что не совпадает ни с $(26, 26)$ для A–B, ни с $(24, 24)$ для B–A. Аномалия сериализации наблюдается.

Уровень SERIALIZABLE. Последовательное выполнение A, затем B.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем SERIALIZABLE.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

Сеанс A. Результат: serializable.

```
SHOW transaction_isolation;
```

Сеанс A. Обновлена одна строка: в IVT-21 записано 26.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-22')
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. A зафиксирована.

```
COMMIT;
```

Сеанс B. Открытие транзакции с уровнем SERIALIZABLE.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

Сеанс B. Результат: serializable.

```
SHOW transaction_isolation;
```

Сеанс B. Прочитано зафиксированное значение IVT-21: 26. В IVT-22 записано 26.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-21')
WHERE "GroupCode" = 'IVT-22';
```

Сеанс B. B зафиксирована.

```
COMMIT;
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 26
 IVT-22    | 26
```

Уровень SERIALIZABLE. Последовательное выполнение B, затем A.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс B. Открытие транзакции с уровнем SERIALIZABLE.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

Сеанс B. Результат: serializable.

```
SHOW transaction_isolation;
```

Сеанс B. Обновлена одна строка: в IVT-22 записано 24.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-21')
WHERE "GroupCode" = 'IVT-22';
```

Сеанс B. B зафиксирована.

```
COMMIT;
```

Сеанс A. Открытие транзакции с уровнем SERIALIZABLE.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

Сеанс A. Результат: serializable.

```
SHOW transaction_isolation;
```

Сеанс A. Прочитано зафиксированное значение IVT-22: 24. B IVT-21 записано 24.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-22')
WHERE "GroupCode" = 'IVT-21';
```

Сеанс A. A зафиксирована.

```
COMMIT;
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 24
```

Уровень SERIALIZABLE. Параллельное выполнение.

Сеанс A. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс B. Завершение возможной предыдущей транзакции. Вне транзакции команда не меняет данные.

```
ROLLBACK;
```

Сеанс A. Очистка тестовой таблицы вне явной транзакции; изменение сразу зафиксировано.

```
TRUNCATE "StudentGroups";
```

Сеанс A. Восстановление исходных данных вне явной транзакции: вставлены и зафиксированы две строки.

```
INSERT INTO "StudentGroups"
    ("Id", "GroupCode", "CourseYear", "StudentCount", "FacultyId")
VALUES
    ('00000000-0000-0000-0000-000000000001', 'IVT-21', 4, 24,
     '00000000-0000-0000-0000-000000000010'),
    ('00000000-0000-0000-0000-000000000002', 'IVT-22', 4, 26,
     '00000000-0000-0000-0000-000000000010');
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 24
 IVT-22    | 26
```

Сеанс A. Открытие транзакции с уровнем SERIALIZABLE.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

Сеанс A. Результат: serializable.

```
SHOW transaction_isolation;
```

Сеанс B. Открытие транзакции с уровнем SERIALIZABLE.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

Сеанс B. Результат: serializable.

```
SHOW transaction_isolation;
```

Сеанс A. Обновлена одна строка: IVT-21 получает 26. A пока не зафиксирована.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-22')
WHERE "GroupCode" = 'IVT-21';
```

Сеанс B. Обновлена другая строка: B читает старое зафиксированное значение IVT-21, равное 24, и записывает 24 в IVT-22. B пока не зафиксирована.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-21')
WHERE "GroupCode" = 'IVT-22';
```

Сеанс A. A успешно зафиксирована.

```
COMMIT;
```

Сеанс B. Фиксация отклонена с SQLSTATE 40001; изменение B отменено.

```
COMMIT;
```

```
ERROR: 40001: could not serialize access due to read/write dependencies among transactions
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 26
 IVT-22    | 26
```

Повтор отменённой B целиком.

Сеанс B. Открытие транзакции с уровнем SERIALIZABLE.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
```

Сеанс B. Результат: serializable.

```
SHOW transaction_isolation;
```

Сеанс B. B новой транзакции прочитано значение 26; в IVT-22 записано 26.

```
UPDATE "StudentGroups" SET "StudentCount" =
    (SELECT "StudentCount" FROM "StudentGroups" WHERE "GroupCode" = 'IVT-21')
WHERE "GroupCode" = 'IVT-22';
```

Сеанс B. Повторная B успешно зафиксирована.

```
COMMIT;
```

Сеанс A. Проверочная выборка после завершения транзакций.

```
SELECT "GroupCode", "StudentCount" FROM "StudentGroups"
ORDER BY "GroupCode";
```

```
 GroupCode | StudentCount
-----------+-------------
 IVT-21    | 26
 IVT-22    | 26
```

Вывод для SERIALIZABLE: фиксация аномалии предотвращена ошибкой 40001. После полного повтора B получен результат $(26, 26)$, соответствующий последовательному порядку A–B.

Триггеры

Создан скрипт `sql/04_triggers.sql` с двумя триггерами:

1. `trg_studentgroup_check_nonneg` проверяет `StudentCount` ≥ 0 перед UPDATE/INSERT и откатывает транзакцию при нарушении.
2. `trg_examsession_teacher_conflict` не позволяет ставить одного преподавателя на две группы одновременно в одну дату.

Продемонстрировано: корректное изменение сохраняет целостность; попытка записи некорректного значения или конфликта вызывает откат транзакции.

Заключение

Собрана база учёта сессии из двух стержневых сущностей и одной ассоциативной. Идентификаторы сущностей — `UUID`, связи идут по ним, а код группы и название дисциплины остаются уникальными естественными ключами. База заполнена и проверена запросами с соединением, CTE, окном и коррелированным подзапросом. В PostgreSQL грязного чтения нет: READ UNCOMMITTED по поведению совпадает с READ COMMITTED. В проверенном сценарии SQL-инкрементов оба изменения сохраняются после ожидания блокировки строки. Неповторяющиеся чтения и фантомы наблюдаются на READ COMMITTED, но отсутствуют на REPEATABLE READ. Аномалия сериализации возникает на REPEATABLE READ; на SERIALIZABLE конфликтующая транзакция отменяется с SQLSTATE 40001 и требует полного повтора. Эти выводы подтверждены выполнением SQL-сценариев в двух независимых сеансах PostgreSQL 16.15. Триггеры проверяют условия, которые не выражаются столбцовым `CHECK`. Роли и представления отрезают пользователю `test` чужие столбцы и чужие таблицы. Функции принимают параметры, проверяют вход и возвращают текст ошибки.